PMS Hotel Boutique

Responsabilidades para cerrar el Frontend Web

Distribucion basada en el backlog original y el ownership definido para WEB-1, WEB-2, WEB-3 y WEB-4

Objetivo: dejar el frontend Web completo, navegable y coherente usando mocks, servicios frontend-first y estados de sesion locales cuando corresponda. En esta etapa no se requiere Backend real, base de datos real, autenticacion real ni pagos reales.

Regla principal: cada integrante debe trabajar dentro de sus responsabilidades originales. Antes de crear algo nuevo, debe revisar lo que ya existe en main y completar/reutilizarlo.

WEB-1 - Booking publico

Responsabilidad original: booking, checkout publico, multi-room y rooms UI publica.

Meta de cierre: una persona debe poder completar una reserva simulada de principio a fin sin encontrar rutas rotas.

Flujo que debe quedar completo

```
Inicio publico
  -> Buscar disponibilidad
  -> Resultados
  -> Detalle de habitacion
  -> Seleccion / multi-room
  -> Ocupantes por habitacion
  -> Datos del huesped
  -> Resumen / review
  -> Garantia o pago simulado
  -> Confirmacion
```

Trabajo concreto

1. Corregir el journey publico y separar rutas publicas de las rutas Staff.
   - La busqueda publica no debe terminar en el board privado de habitaciones.
   - Verificar que todos los enlaces publicos existan y sean coherentes.
2. Completar resultados de disponibilidad (IMP-WEB-0103).
   - Fechas, huespedes, tipos de habitacion, precio, capacidad y amenities resumidas.
   - Estados loading, empty, error y retry.
3. Completar Room Type / detalle de habitacion.
   - Nombre, descripcion, imagen o galeria, capacidad, camas, amenities, precio y restricciones frontend.
   - Accion clara para seleccionar.
4. Completar Add/select room.
   - Seleccionar una habitacion desde resultados o detalle.
   - Volver atras sin perder la seleccion local cuando sea necesario.
5. Completar carrito multi-room (IMP-WEB-0105).
   - Una Reservation puede contener varios Stay.
   - Cada seleccion debe mantener tipo de habitacion, fechas, huespedes, ocupantes y precio mock.
6. Implementar ocupantes por habitacion.
   - No manejar solamente una cantidad global de huespedes.
   - Cada Stay debe conocer sus ocupantes segun el contrato frontend.
7. Completar datos del huesped.
   - Formulario con validacion frontend y mensajes accesibles.
   - Consumir identidad de WEB-2 si existe; no duplicar Account/Auth.
8. Completar garantia/pago visual.
   - Pantalla y estados frontend simulados.
   - No implementar reglas internas de pagos; esas pertenecen a WEB-4.
9. Completar review de reserva.
   - Habitaciones, ocupantes, fechas, total frontend, datos del huesped y politicas resumidas.

10. Completar confirmacion final.
    - Codigo mock, resumen y acciones posteriores.
    - No afirmar persistencia real.
11. Fijar contrato frontend de Reservation multi-room (IMP-WEB-0111).
    - Definir la forma que Booking necesita para representar Reservation y sus Stay.
    - No disenar Backend.
12. Cerrar QA Booking publico (IMP-WEB-0113).
    - Probar el journey completo de Search a Confirmation.
    - Confirmar rutas, estados y ausencia de links rotos.

No debe apropiarse de

- Reservations Staff
- Housekeeping
- Folio privado
- Revenue
- Rates engine interno
- Security
- Multi-property
- Groups
- Integrations

Dependencias

- Consume Availability/Rates de WEB-4.
- Puede consumir identidad/sesion frontend de WEB-2.
- Si modifica conceptos compartidos de Availability, Rates o Payments, debe coordinar con WEB-4.

WEB-2 - Identity / Account / Security / Multi-property

Responsabilidad original: auth, account, profile, rewards, promotions, privacy, permissions, security y properties.

Meta de cierre: Account/Identity/Security/Multi-property deben funcionar como una experiencia frontend coherente y consumir las capas existentes en lugar de usar hardcodes directos en las paginas.

Trabajo concreto

13. Terminar acceso Guest final (IMP-WEB-0202).
    - Flujo frontend simulado: acceso -> loading -> error o success -> Account.
    - No requiere JWT ni Backend.
14. Unificar la sesion/auth frontend.
    - Evitar booleanos o estados independientes por pagina.
    - Mantener una sola autoridad mock para signed in / signed out y, si aplica, session Staff demo.
15. Conectar Account con sus capas existentes.
    - Mock Service -> DTO -> Mapper -> Domain -> Query/Hook -> UI.
    - Eliminar datos hardcodeados que dupliquen una fuente ya existente.
16. Completar Account Dashboard.
    - Identidad, accesos a Profile, historial, Rewards, Promotions, facturas/acciones cuando correspondan y configuracion.
17. Completar Profile.

- Visualizacion, edicion de campos permitidos, loading, error, save, success, dirty state y cancel/revert.

18. Completar Reservation History.
    - Reservas actuales y pasadas, estado, fechas, property y detalle.
    - La semantica Reservation/Stay debe revisarse con WEB-3.
19. Completar Rewards.
    - Balance/puntos, tier, beneficios, empty/error/loading e historial si corresponde.
    - No calcular puntos reales.
20. Completar Promotions.
    - Listado, detalle, elegibilidad visual, vigencia y estados.
    - No implementar motor comercial real.
21. Completar Privacy / Consent (IMP-WEB-0904).
    - Consentimientos, finalidad, estado, activar/desactivar y feedback.
    - Persistencia frontend/mock.
22. Completar Security / Sessions.
    - Sesiones activas simuladas, logout, MFA UI y estados de seguridad.
    - Sin auth real de servidor.
23. Completar Roles / Permissions UI (IMP-WEB-0901).
    - Roles, permisos, matrices/listas y edicion simulada.
    - Sin enforcement Backend.
24. Completar Property Switcher.
    - Selector frontend de propiedad y cambio de contexto mock.
25. Completar Multi-property.
    - Dashboard, selector, busqueda, metricas por property, loading/empty/error.
    - Si necesita metricas de Revenue o Availability, coordinar con WEB-4.
26. Participar en el shell privado desde la frontera de identidad.
    - Session, usuario, rol, property y logout.
    - No apropiarse de los modulos funcionales de WEB-3 o WEB-4.
27. Cerrar QA Identity/Account (IMP-WEB-0210).
    - Access, Account, Profile, History, Rewards, Promotions y Privacy.

No debe apropiarse de

- Reservation lifecycle
- Room move
- Housekeeping
- Groups
- Folio
- Payment business rules
- Availability engine
- Revenue calculations

Dependencias y revisiones

- Reservation History consume semantica de Reservation/Stay de WEB-3.
- Multi-property puede consumir metricas de WEB-4.
- Auth/Session/Property context sirve como base para las areas privadas de WEB-3 y WEB-4.

WEB-3 - Reservations / Operations / B2B / Integrations / Reporting

Responsabilidad original: reservations, stays, housekeeping, maintenance, companies, agencies, groups, integrations y reports.

Meta de cierre: convertir lo ya implementado en main en un PMS Staff navegable y compuesto, evitando crear foundations duplicadas.

Trabajo concreto

28. Completar Reservation Center.
    - Lista, filtros, busqueda y detalle con navegacion real.
29. Completar Reservation Detail.
    - Guest, Stay, fechas, estado, room assignment, enlaces a Folio/Payment cuando corresponda y acciones frontend.
30. Cerrar cancelacion.
    - Motivo -> confirmacion -> mutation mock -> success/error.
31. Cerrar No-show.
    - Journey completo desde detalle, sin dejarlo como componente aislado.
32. Cerrar Waitlist.
    - Lista -> detalle -> disponibilidad mock -> convertir -> resultado.
33. Implementar Calendar/Gantt (IMP-WEB-0308).
    - Habitaciones, fechas, reservas, ocupacion, seleccion y navegacion temporal.
34. Completar Room Move.
    - Stay actual -> seleccionar room -> confirmar -> mutation mock -> actualizar UI.
    - Coordinar impacto de Availability con WEB-4.
35. Completar Stay Extension.
    - Extender fecha -> disponibilidad mock -> revisar -> confirmar.
36. Completar Housekeeping Staff.
    - Board, estados, habitaciones, readiness y acciones.
37. Completar Housekeeping discrepancies.
    - Comparacion Front Office vs Housekeeping y flujo de resolucion.
38. Completar Inspection Reject.
    - Inspeccionar -> rechazar -> motivo -> volver a limpieza cuando aplique.
39. Completar Maintenance.
    - Ordenes, activo, prioridad, lifecycle e historial mock.
40. Completar OOO/OOS.
    - Room -> Out of Order / Out of Service -> motivo -> periodo -> confirmar.
    - Coordinar efectos de inventory con WEB-4.
41. Completar Companies.
    - Listado, detalle, datos comerciales, contactos y contrato visual.
42. Completar Agencies.
    - Profile, comision, voucher y datos de agencia.
43. Completar Groups lifecycle (IMP-WEB-0705).
    - Group, block, fechas, room block, pickup, rooming list y estado.
44. Completar Integration Center (IMP-WEB-0801).
    - Integraciones, health, estado, ultima sincronizacion y capabilities mock.

45. Completar Error Queue.
    - Listado, detalle, filtros, status y retry simulado.
46. Completar Reports.
    - Listados, filtros, saved reports/scheduler si aplica y export simulado.
47. Asegurar rutas y navegacion Staff.
    - Ningun modulo importante debe quedar solo como component/service/hook sin pagina usable.
48. Cerrar QA por journeys.
    - Reservations -> detail -> cancel/no-show/waitlist.
    - Rooms -> housekeeping/maintenance.
    - Groups -> block/pickup.
    - Integrations -> error queue/retry.

No debe apropiarse de

- Guest Account
- Auth/Security
- Folio
- Payments
- Rates
- Revenue
- Booking publico

Dependencias y revisiones

- Room Move, Stay Extension, OOO/OOS y Groups pueden afectar Availability/Inventory de WEB-4.
- Reservation History de WEB-2 debe consumir sus conceptos de Reservation/Stay.
- Permisos/seguridad de sus modulos deben revisarse con WEB-2.

WEB-4 - Folio / Payments / Commercial / Revenue

Responsabilidad original: folio, payments, receivables, availability, rates, inventory, revenue y channels.

Meta de cierre: dejar el nucleo comercial/financiero frontend navegable, con una fuente mock coherente y APIs publicas consumibles por Booking y Operations.

Trabajo concreto

49. Priorizar Availability.
    - Fuente mock unica para fecha, room type, availability, rate y restrictions.
    - Debe servir como dependencia estable para WEB-1.
50. Implementar Matriz ATS (IMP-WEB-0601).
    - Fechas, room types, disponibilidad, ocupacion, sold y remaining.
51. Completar Restrictions.
    - MinLOS, CTA, CTD, closed/open y sell limit.
52. Completar Rates.
    - Rate plans, precio, vigencia, restricciones, canales y detalle.
53. Completar Inventory.
    - Room type, availability, blocks, out of order y overrides mock.
54. Completar Overbooking.
    - Limite, estado, warning y confirmacion frontend.

55. Exponer API publica frontend para WEB-1.
    - Booking no debe leer fixtures internos de WEB-4.
    - Exponer funciones/hook publicos para availability, rates y restrictions.
56. Completar Folio avanzado (IMP-WEB-0401).
    - Ruta/pagina real con detalle, cargos, balance, status y routing.
57. Completar Charge Routing.
    - Cargo -> folio destino -> regla -> guardar.
58. Completar Split.
    - Division visual de cargos por folio, guest o company.
59. Completar Transfer.
    - Mover cargos entre folios frontend/mock.
60. Completar Payment Detail.
    - Amount, method, status y transaction mock.
61. Completar Authorization.
    - Authorize -> pending -> approved/declined mock.
62. Completar Capture / Void / Refund.
    - Confirmacion, error, success e historial local simulado.
63. Completar Receivables.
    - Saldo, company/agency, aging mock y status.
64. Completar Revenue Dashboard.
    - Occupancy, ADR, RevPAR, pickup, pace y forecast con datos mock consistentes.
65. Completar Channel Mix y Channels.
    - Canal, bookings, revenue mock, porcentaje, status y mapping visual si aplica.
66. Asegurar rutas reales.
    - Availability, Rates, Inventory, Folio, Payments y Revenue deben estar montados en paginas del PMS privado.
67. Cerrar QA comercial/financiera.
    - Availability -> Rate -> restriction.
    - Folio -> routing/split/transfer.
    - Payment -> auth/capture/void/refund.
    - Revenue -> forecast/pickup/channel mix.

No debe apropiarse de

- Booking UI publica
- Guest Account
- Reservations lifecycle
- Housekeeping
- Groups
- Security

Dependencias y revisiones

- Availability/Rates deben quedar disponibles para WEB-1.
- Reservation/Stay/Groups de WEB-3 pueden tener efectos que deban reflejarse en Availability/Folio/Revenue.
- Multi-property de WEB-2 puede consumir metricas comerciales agregadas.

Dependencias entre integrantes

```
WEB-4 Availability / Rates
        -> WEB-1 Booking publico

WEB-2 Session / Identity / Property context
        -> WEB-3 y WEB-4 Private Web

WEB-3 Reservation / Stay semantics
        -> WEB-2 Reservation History

WEB-3 Room / Stay / Groups
        <-> WEB-4 Availability / Folio / Revenue
```

Regla de trabajo para todos

68. Revisar que existe en main.
69. Revisar la API publica del modulo.
70. Revisar service, mapper y hook existentes.
71. Revisar la route/pagina existente.
72. Completar lo faltante.
73. Crear codigo nuevo solo cuando realmente no exista una pieza reutilizable.

Trabajo compartido de cierre

- Shell privado real: sidebar, header, navegacion y property context coherentes.
- Eliminar demos aisladas: si existe service/mapper/hook, la UI debe consumirlo.
- Rutas reales: evitar modulos importantes sin pagina o sin acceso desde navegacion.
- Estados consistentes: loading, error, empty y retry donde corresponda.
- QA final de journeys principales.
- Evitar links rotos y rutas que mezclen superficie publica con Staff.

Criterio para considerar el Frontend Web fijado

Integrante	Resultado minimo de cierre
WEB-1	Booking publico completo y navegable desde busqueda hasta confirmacion.
WEB-2	Account/Auth/Security/Multi-property coherentes y sin hardcodes directos en paginas.
WEB-3	Reservations/Operations/B2B/Integrations navegables y compuestos como PMS Staff.
WEB-4	Availability/Folio/Payments/Revenue navegables, con mocks coherentes y APIs publicas.

Todos los integrantes deben mantener su ownership original y coordinar cambios cross-module mediante los reviewers definidos en el backlog. La meta de esta fase es cerrar journeys frontend completos con mocks coherentes, no implementar Backend real.